06-5-admission-rollout

accounts-api security assessment · generated from the repository, not hand-edited

> **Section 6 supporting write-up: audit/enforce designation per policy, what triggers promotion, and what counts as evidence that a gate is ready.**
>
> Assembled by `scripts/build-submission.py` from the repository. Source paths are given above each section.

`docs/05-admission-rollout.md`

Section 5 — admission policy, and the rollout problem

A cluster shared with a dozen other teams is a rollout problem as much as a policy problem. The policy that is technically correct and breaks four other teams on a Tuesday gets the admission controller uninstalled, and then the correct policy protects nothing. So every policy carries its day-one mode and its promotion criteria **in its own annotations**, next to the rule, not in a wiki page nobody opens.

Day-one posture

policy	day one	scope	what triggers promotion
`disallow-host-namespaces`	**Enforce**	cluster (minus 5 infra namespaces)	already promoted. 14-day dry-run found 2 violations, both in `kube-system`. This one is non-negotiable because it is load-bearing for every NetworkPolicy in the cluster.
`disallow-hostpath`	**Enforce**	cluster (minus infra)	already promoted. Same dry-run: violations only in CSI/CNI daemonsets, excluded by **namespace** rather than by path — a path allowlist is a bypass waiting to be found.
`restrict-rbac-wildcards`	**Enforce**	cluster (minus named operator ClusterRoles)	already promoted. 4 violations, all Helm-installed operator ClusterRoles, each excluded by name — and each name is a line somebody had to write and review.
`restrict-privileged-rolebindings`	**Enforce**	cluster (minus infra + one named break-glass binding)	already promoted, and it ships *with* `restrict-rbac-wildcards` rather than after it. Reason: the wildcard policy audits the Role and says nothing about the RoleBinding, so a tenant who cannot write `verbs: ["*"]` can still bind their ServiceAccount to the `cluster-admin` ClusterRole that Kubernetes ships. Landing the first policy without the second would have published the bypass.
`restrict-registries`	**Enforce**	cluster	already promoted. The allowlist was derived from a 30-day inventory of every image reference actually running, so it was empirically complete *before* it was enforced. That order — enumerate reality, then enforce the enumeration — is the pattern for every promotion below.
`require-restricted-`	**Audit** cluster-wide, **Enforce** in		14 consecutive days of zero PolicyReport failures + acknowledgement from each of the 12 service owners +

policy	day one	scope	what triggers promotion
`securitycontext`	`ns/accounts`		the two known offenders (a legacy JVM in `ns/payments` needing `CAP_NET_BIND_SERVICE`, a vendor OTel collector running as root) migrated or holding a namespace-scoped exception PR. Target 4 weeks.
`require-image-digest`	**Audit** cluster-wide, **Enforce** in `ns/accounts`		every team's CD tooling must resolve tags at build time rather than apply time. Argo image-updater and Flux do; hand-rolled `kubectl` pipelines mostly do not. Evidence: 14 days zero failures **per namespace**, promoted namespace by namespace.
`verify-image-signature`	**Audit** cluster-wide, **Enforce** in `ns/accounts`		30 days of Audit showing 100% of admitted images in a namespace carry a verifiable signature. Promoted one namespace at a time — a single cluster-wide flip would break twelve teams at once, and the resulting incident review would end with the policy removed, which is a worse outcome than an unsigned image.
`disallow-secrets-as-env`	**Audit** cluster-wide, **Enforce** in `ns/accounts`		longest runway: `envFrom: secretRef` is in 14 of ~40 workloads and fixing it is an *application* change (read a file, re-read on change), not a manifest change. Needs the CSI driver installed cluster-wide and a published migration recipe first. Estimated two quarters.
`require-resource-limits`	**Audit**	cluster	deliberately not promoted early. This is a tenancy control, not a security control, and I am not spending enforcement credibility on it. A per-namespace `LimitRange` with defaults is the better fix and makes the policy nearly redundant.

The pattern. I enforce in `ns/accounts` — the namespace I own — on day one for every policy, including the ones that are cluster-wide Audit. That is not just tidiness: it is the evidence I take to the other twelve teams. "We have run this in enforce for six weeks on the service that holds the customer ledger, here is the incident count (zero) and here is the migration diff" is an argument. "Policy says so" is not, and platform teams that lead with the second one get their webhooks removed.

What counts as evidence to promote, generally: a Kyverno `PolicyReport` with zero failures for the stated window, across the namespaces in scope, *with* representative deploy traffic in that window — a quiet fortnight over the December freeze is not evidence. I check the second condition by counting admission requests in the same period, not just failures.

Failure mode I designed around

Kyverno's webhook is in the admission path. If it is unavailable and `failurePolicy: Fail`, nothing deploys cluster-wide, including the fix for whatever broke Kyverno. If it is `Ignore`, an attacker who can crash the webhook has disabled every policy here.

I ship `failurePolicy: Fail` for the `Enforce` policies with the webhook running 3 replicas across zones and a PodDisruptionBudget, and I accept the availability risk — because `Ignore` turns "the admission controller is down" into a silent security event, and the whole point of section 5 is to not have silent security events. The

compensating control for the availability side is that `disallow-host-namespaces` and friends are also checked in CI (`conftest` against the rendered manifests), so the common case — a bad manifest — is caught before it ever reaches the cluster, and the webhook is the backstop rather than the only gate.

Running the tests

```
kyverno test policy/kyverno/tests          # 39 assertions, three fixtures
./scripts/run-policy-tests.sh              # the above plus conftest + waivers
```

Captured output is in `../test-output/`.

`verify-image-signature` is **not** in the `kyverno test` suite. The CLI cannot verify a Sigstore signature without registry and Rekor access, and a fixture that asserts `pass` for it would be a lie in a file whose entire job is to be trustworthy. It is exercised by `./scripts/run-policy-tests.sh --integration`, which needs a kind cluster and network.

Two blind spots I went looking for, and closed

Writing the policies was the easy half. These are the two places where the rule set *looked* complete and was not, both found by trying to defeat it rather than by reading it — which is why `bypass-workload.yaml` exists as a first-class fixture.

- **`initContainers`.** Kyverno's autogen rewrites `spec.containers` for controllers, so a rule written against `containers` reads correctly and never looks at `initContainers`. A privileged init step that drops a setuid binary into a shared `emptyDir` and exits is admitted, and it has finished its work before the main container starts. Closed by `init-containers-restricted`, with the bypass fixture carrying exactly that shape.
- **`RoleBinding`.** Covered above; the short version is that the wildcard lives in a ClusterRole Kubernetes ships, so a policy that only inspects Roles can be satisfied without giving up any privilege at all.

Both are now assertions in the suite, so they cannot silently regress.